

CSE 4111

Compiler Design

Sakurah Ismail

Lecturer

Department of Computer Science & Engineering
North Bengal International University

FINITE AUTOMATA

- Finite automata is **the simplest machine to recognize patterns**. The finite automata or finite state machine is an abstract machine that has five elements or tuples. It has a set of states and rules for moving from one state to another but it depends upon the applied input symbol.
- **There are two types of finite automata:**
 - DFA (deterministic finite automata)
 - NFA (non-deterministic finite automata)

Q: What is finite automata? Why we use it? Also write down the types of it.

Applications of Finite Automata

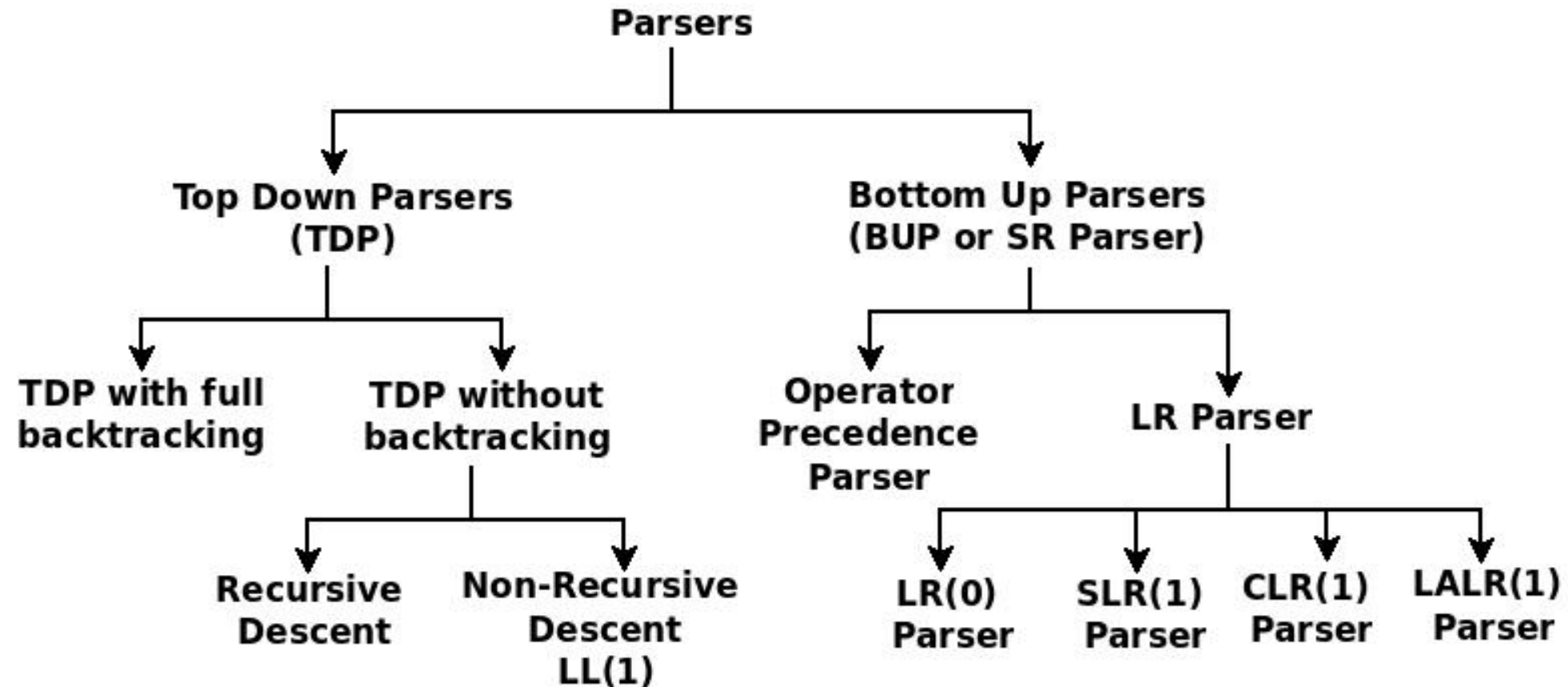
- For the designing of lexical analysis of a compiler.
- For recognizing the pattern using regular expressions.
- For the designing of the combination and sequential circuits.
- Used in text editors.
- For the implementation of spell checkers.

Q: Write the applications of finite automata.

Parser

- The parser is that phase of the compiler which takes a token string as input and with the help of existing grammar, converts it into the corresponding Intermediate Representation.
- Parsing breaks down words into functional units that can be converted into machine language.
- The parser is also known as **Syntax Analyzer**.

Types of Parser



Top Down Parser

- **Working of Top Down Parser :**

Let's consider an example where grammar is given and you need to construct a parse tree by using top down parser technique.

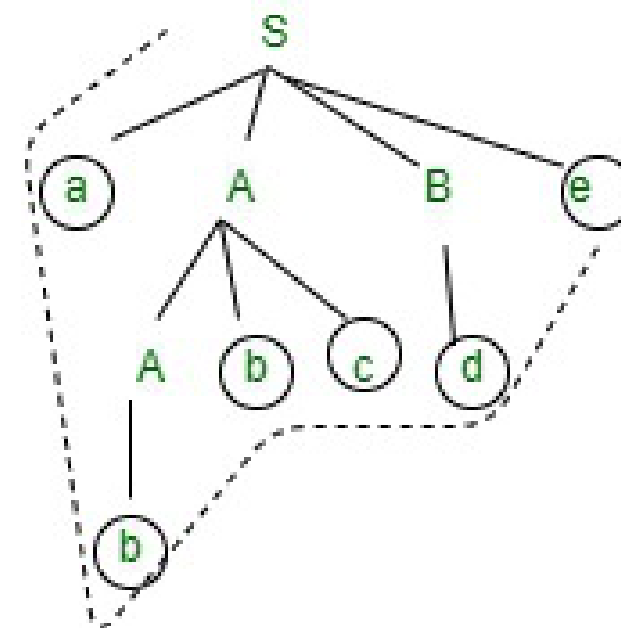
String: abbcde

- **Example –**

$S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$



Q: Write short notes on top-down parser.

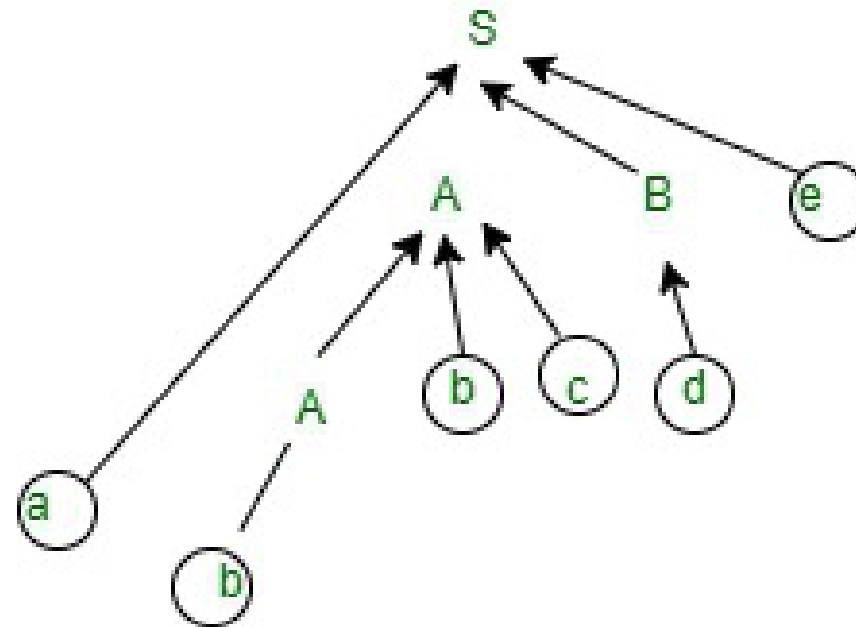
Bottom Up Parser

- Example –

$S \rightarrow aABe$; $A \rightarrow Abc \mid b$; $B \rightarrow d$

Now, let's consider the input to read and to construct a parse tree with bottom-up approach.

- First, you can start with $A \rightarrow b$.
- Now, expand $A \rightarrow Abc$.
- After that Expand $B \rightarrow d$.
- In the last, just expand the $S \rightarrow aABe$
- Final string, you will get **abbcde**.



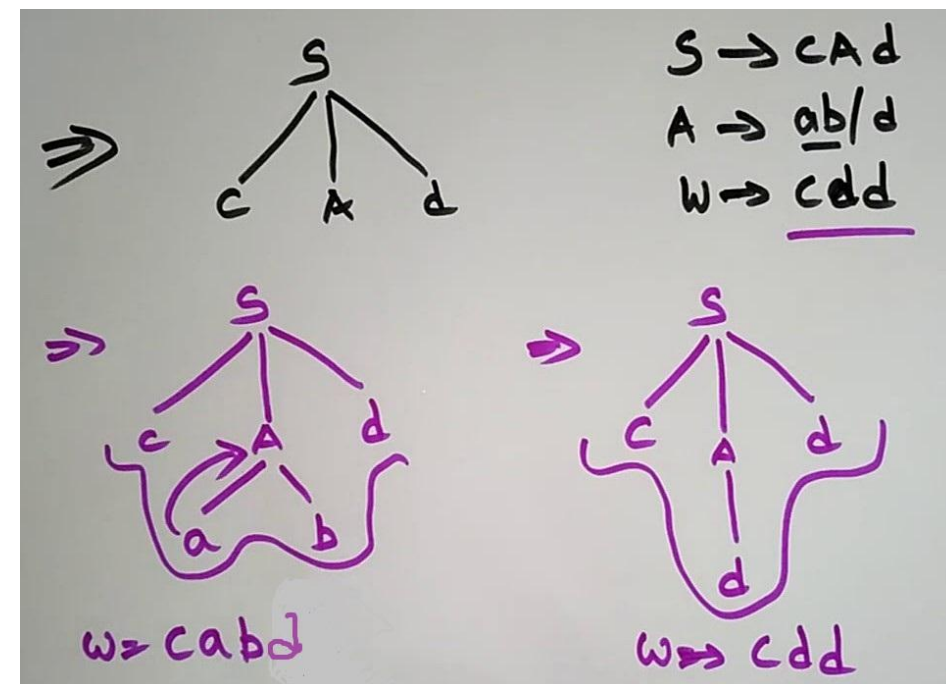
Q: Write short notes on bottom-up parser.

Recursive-Descent Parsing (Backtracking)

Backtracking : It means, if one derivation of a production fails, the syntax analyzer **restarts the process** using different rules of same production. This technique may process the input string more than once to determine the right production.

Q: Write short notes on recursive-descent parser.

Q: What is backtracking in parsing?



Intermediate Code generation Techniques

- Intermediate code is a **temporary, simplified version of the program** that sits between source code and machine code. The intermediate code generator, which is usually the front end of a compiler, is used to generate intermediate code.

The following are commonly used intermediate code representation:

- Postfix Notation
- Three-Address Code
- Syntax Tree

Notation / Polish Notation

- Polish notation is a **notation form for expressing arithmetic, logic and algebraic equations**. Its most basic distinguishing feature is that operators are placed on the left of their operands. If the operator has a defined fixed number of operands, the syntax does not require brackets or parenthesis to lessen ambiguity.

Example:

Infix notation with parenthesis: $(3 + 2) * (5 - 1)$

Polish notation: $* + 3 2 - 5 1$

When used as the syntax for programming language interpreters, Polish notation can be readily parsed into an abstract syntax tree and stored in a stack. In traditional infix notation with brackets, the equation has to be parsed, the brackets removed, and the operator and operands repositioned.

Types of Notation

There are three types of notation—

- i. Infix notation
- ii. Postfix notation
- iii. Prefix notation

Operators Precedence

OPERATOR	PRECEDENCE	VALUE
Exponentiation (\$ or ↑ or ^)	Highest	3
*, /	Next highest	2
+, -	Lowest	1

Notations



infix

$(a + b) * c$

$a + (b * c)$

postfix

$a b + c *$

$a b c * +$

prefix

$* + a b c$

$+ a * b c$

Infix form : $\langle identifier \rangle \langle operator \rangle \langle identifier \rangle$

Postfix form : $\langle identifier \rangle \langle identifier \rangle \langle operator \rangle$

Prefix form : $\langle operator \rangle \langle identifier \rangle \langle identifier \rangle$

Notations (Contd.)

Examples of infix to prefix and post fix

Infix	PostFix	Prefix
$A+B$	$AB+$	$+AB$
$(A+B) * (C + D)$	$AB+CD+*$	$*+AB+CD$
$A-B/(C*D^E)$	$ABCDE^*/-$	$-A/B*C^DE$

Problems on notations

Postfix Expression	Infix Equivalent	Result
4 5 7 2 + - ×	$4 \times (5 - (7 + 2))$	-16
3 4 + 2 × 7 /	$((3 + 4) \times 2) / 7$	2
5 7 + 6 2 - ×	$(5 + 7) \times (6 - 2)$	48
4 2 3 5 1 - + × + ×	$? \times (4 + (2 \times (3 + (5 - 1))))$	not enough operands
4 2 + 3 5 1 - × +	$(4 + 2) + (3 \times (5 - 1))$	18
5 3 7 9 + +	$(3 + (7 + 9)) \dots 5???$	too many operands

Stack Representation

Infix Expression : A+B*(C^D-E)				
Token	Action	Result	Stack	Notes
A	Add A to the result	A		
+	Push + to stack	A	+	
B	Add B to the result	AB	+	
*	Push * to stack	AB	* +	* has higher precedence than +
(	Push (to stack	AB	(* +	
C	Add C to the result	ABC	(* +	
^	Push ^ to stack	ABC	^ (* +	
D	Add D to the result	ABCD	^ (* +	
-	Pop ^ from stack and add to result	ABCD^	(* +	- has lower precedence than ^
	Push - to stack	ABCD^	- (* +	
E	Add E to the result	ABCD^E	- (* +	
)	Pop - from stack and add to result	ABCD^E-	(* +	Do process until (is popped from stack
	Pop (from stack	ABCD^E-	* +	
	Pop * from stack and add to result	ABCD^E-*	+	Given expression is iterated, do Process till stack is not Empty, It will give the final result
	Pop + from stack and add to result	ABCD^E-*+		
Postfix Expression : ABCD^E-*+				

Infix to prefix

$(a + (b * c) / (d - e)) = +a/*bc-de$

NOTE: scan the infix string in reverse order.

SYMBOL	PREFIX	OPSTACK
)	Empty	)
)	Empty	)
e	e	)
-	e	)
d	de	)
(	-de	)
/	-de	)
)	-de	)
c	c-de	)
*	c-de	)
b	bc-de	)
(	*bc-de	)
+	/*bc-de	)
a	a/*bc-de	)
(	+a/*bc-de	Empty

Infix 2 Prefix Stack rep.

Infix Expression: $(P + (Q * R) / (S - T))$

Note: - Read the infix string in reverse

Symbol Scanned	Stack	Output
)	)	-
)	))	-
T	))	T
-	))-	T
S	))-	ST
(	)	-ST
/	)/	-ST
)	)/)	-ST
R	)/)	R-ST
*	)/)*	R-ST
Q	)/)*	QR-ST
(	)/	*QR-ST
+	)+	/+QR-ST
P	)+	P/+QR-ST
(	Empty	+P/+QR-ST

Prefix Expression: $+P/+QR-ST$

Thank You